

Debugging JavaScript Review

Common Types of Error Messages

- **SyntaxError:** These errors happen when you write something incorrectly in your code, like missing a parenthesis, or a bracket. Think it like a grammar mistake in a sentence.

```
const arr = ["Beau", "Quincy" "Tom"]
```

- **ReferenceError:** There are several types of Reference Errors, triggered in different ways. The first type of reference error would be defined variables. Another example of a ReferenceError is trying to access a variable, declared with `let` or `const`, before it has been defined.

```
1 console.log(num);  
2 const num = 50;
```

- **TypeError:** These errors occur when you try to perform an operation on the wrong type.

```
const developerObj = {  
  name: "Jessica",  
  country: "USA",  
  isEmployed: true  
};
```



```
const arr = [];  
arr.length = -1;
```

The `throw` Statement

- **Definition:** The `throw` statement in JavaScript is used to throw a user defined exception. An exception in programming, is when an unexpected event happens and disrupts the normal flow of the program.

```
1 function validateNumber(input) {  
2   if (typeof input !== "number") {  
3     throw new TypeError("Expected a number, but received " + typeof input);  
4   }  
5   return input * 2;  
6 }  
7  
8 console.log(validateNumber("Naomi")); // TypeError: Expected a number,
```

```
"TypeError: Expected a number, but received string  
at validateNumber (https://2-19-8-sandpack.codesandbox.io/index.js:11:11)  
at $csb$eval (https://2-19-8-sandpack.codesandbox.io/index.js:16:13)  
at J (https://2-19-8-sandpack.codesandbox.io/static/js/sandbox.8a7d01a44.js:1:159097)  
at K.evaluate (https://2-19-8-sandpack.codesandbox.io/static/js/sandbox.8a7d01a44.js:1:171816)  
at je.evaluateTranspiledModule (https://2-19-8-sandpack.codesandbox.io/static/js/sandbox.8a7d01a44.js:1:18219  
7)  
at je.evaluateModule (https://2-19-8-sandpack.codesandbox.io/static/js/sandbox.8a7d01a44.js:1:181696)  
at https://2-19-8-sandpack.codesandbox.io/static/js/sandbox.8a7d01a44.js:1:354442  
at Generator.next (<anonymous>)
```

`try...catch...finally`

- **Definition:** The `try` block is used to wrap code that might throw an error. It acts as a safe space to try something that could fail. The `catch` block captures and handles errors that occur in the try block. You can use the error object inside catch to inspect what went wrong. The `finally` block runs after the try and catch blocks, regardless of whether an error occurred. It's commonly used for cleanup tasks, such as closing files or releasing resources.

```
1 function processInput(input) {  
2   if (typeof input !== "string") {  
3     throw new TypeError("Input must be a string.");  
4   }  
5  
6   return input.toUpperCase();  
7 }  
8  
9 try {  
10  console.log("Starting to process input...");  
11  const result1 = processInput("hello");  
12  console.log("Processed result:", result1); // HELLO  
13  const result2 = processInput(9); // throws TypeError  
14
```

"Starting to process input..."

"Processed result:"

"HELLO"

"Error occurred:"

"Input must be a string."

Debugging Techniques

- **debugger** Statement: This statement lets you pause your code at a specific line to investigate what's going on in the program.

```
1 let firstNumber = 5;
2 let secondNumber = 10;
3
4 debugger; // Code execution pauses here
5 let sum = firstNumber + secondNumber;
6
7 console.log(sum);
```

15

- **Breakpoints:** Breakpoints let you pause the execution of your code at a specific line of your choice. After the pause, you can inspect variables, evaluate expressions, and examine the call stack.
- **Watchers:** Watch expressions lets you monitor the values of variables or expressions as the code runs even if they are out of the current scope.



```
console.dir(document);
```

- `console.table()`: This method displays tabular data as a table in the console. It takes one mandatory argument, which must be an array or an object, and one optional argument to specify which properties (columns) to display.

Assignment

- ☐ Review the Debugging JavaScript topics and concepts.

Submit

Ask for Help